Teach Module 1: JVM Architecture and Runtime Model

This README captures the Module 1 teaching session, practice exercises, and hands-on lab.

The original bootcamp document was used only to identify the Module 1 topic. The explanations below are written as independent teaching material.

Module Goal

By the end of this module, you should be able to explain:

```
Java source code -> bytecode -> class loading -> stack/heap memory -> execution -> garbage collection
```

1. What The JVM Does

When you write Java, you do not usually compile directly into machine code for Windows, Linux, or macOS. Instead, Java source code is compiled into bytecode, and bytecode runs inside the JVM, the Java Virtual Machine.

```
Hello.java -> javac -> Hello.class -> JVM -> machine execution
```

The same `.class` file can run on different operating systems as long as that system has a compatible JVM.

2. Source Code, Bytecode, And Execution

Example Java program:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
```

Compile it:

```
javac Hello.java
```

That creates:

```
Hello.class
```

The `.class` file contains bytecode, which is an intermediate instruction format understood by the JVM.

Run it:

```
java Hello
```

At that point, the JVM loads the class, checks it, prepares memory, and executes the `main` method.

3. Main Parts Of The JVM

Think of the JVM as having these major responsibilities:

```
Class Loader
Bytecode Verifier
Runtime Memory Areas
Execution Engine
Garbage Collector
```

The class loader loads `.class` files into memory when the program needs them.

The bytecode verifier checks that bytecode is valid and safe to run.

Runtime memory areas store objects, method calls, variables, class metadata, and execution state.

The execution engine runs the bytecode.

The garbage collector automatically cleans up objects that are no longer reachable.

4. Class Loading

Java does not necessarily load every class at startup. Classes are usually loaded when first needed.

```
User user = new User();
```

If the JVM has not loaded `User.class` yet, it locates and loads it.

Class loading has three broad stages:

```
Loading -> Linking -> Initialization
```

Loading means finding the class file and bringing it into JVM memory.

Linking means verifying and preparing the class.

Initialization means running static initialization logic, such as static fields or static blocks.

```
class AppConfig {
    static String appName = "Banking App";

    static {
        System.out.println("AppConfig loaded");
    }
}
```

The static block runs when the class is initialized.

5. Stack Memory

The stack stores method calls and local variables.

Every time a method is called, Java creates a new stack frame.

```
public class Calculator {
    public static void main(String[] args) {
        int result = add(10, 20);
        System.out.println(result);
    }

    static int add(int a, int b) {
        int sum = a + b;
        return sum;
    }
}
```

When `main` runs, it gets a stack frame.

When `add(10, 20)` is called, `add` gets its own stack frame.

Inside the `add` frame, Java stores:

```
a = 10
b = 20
sum = 30
```

When `add` finishes, its stack frame disappears.

6. Heap Memory

The heap stores objects created with `new`.

```
User user = new User("Asha");
```

The `User` object lives on the heap.

The variable `user` is a reference. If it is a local variable, that reference is stored in the stack, but the actual object is stored in the heap.

```
Stack:
user -> reference/address

Heap:
User object { name = "Asha" }
```

This code creates one object, with two references pointing to it:

```
User user1 = new User("Asha");
User user2 = user1;
```

7. Stack Vs Heap

Stack	Heap
Stores method calls and local variables	Stores objects
Fast allocation and cleanup	Managed by garbage collector
Data disappears when method exits	Objects live while reachable
Thread-specific	Shared across threads

Example:

```
public void processOrder() {
    int quantity = 5;
    Order order = new Order();
}
```

`quantity` is a local primitive value, so it is stored in the stack frame.

`order` is a local reference stored in the stack frame.

The actual `Order` object is stored in the heap.

8. Garbage Collection

Java automatically removes heap objects that can no longer be reached.

```
public void createUser() {
    User user = new User("Ravi");
}
```

When `createUser()` finishes, the local variable `user` disappears from the stack. If nothing else references that `User` object, it becomes eligible for garbage collection.

Garbage collection is not immediate. The object becomes eligible for cleanup, but the JVM decides when cleanup actually happens.

9. Execution Engine And JIT

The JVM can interpret bytecode, but repeatedly interpreting code can be slower.

Modern JVMs use a JIT compiler, meaning Just-In-Time compiler.

The JIT watches which parts of your code run often. Frequently used code paths are compiled into optimized native machine code while the program is running.

That is why long-running Java applications, such as backend services, can become faster after warming up.

10. Why This Matters

Understanding the JVM helps you debug real problems.

If you see:

```
StackOverflowError
```

It often means method calls went too deep, usually because of uncontrolled recursion.

```
public void repeat() {
    repeat();
}
```

If you see:

```
OutOfMemoryError
```

It often means the heap is full, possibly because the application is holding references to too many objects.

```
List<byte[]> data = new ArrayList<>();

while (true) {
    data.add(new byte[1024 * 1024]);
}
```

That list keeps references to every allocated byte array, so the garbage collector cannot clean them.

Quick Check

Answer these mentally:

1. What does `javac` produce?
2. Where do objects created with `new` live?
3. Where are method calls tracked?
4. What is the job of the garbage collector?
5. Why does the JVM use JIT compilation?

Mini Practice

Predict what happens here:

```
class Person {
    String name;

    Person(String name) {
        this.name = name;
    }
}

public class Demo {
    public static void main(String[] args) {
        Person p1 = new Person("Maya");
        Person p2 = p1;
```

```
        p2.name = "Leah";

        System.out.println(p1.name);
    }
}
```

Answer:

```
Leah
```

Why? Because `p1` and `p2` refer to the same object on the heap.

Practice Exercises

Exercise 1: Compile And Inspect Bytecode

Create `HelloJvm.java`:

```
public class HelloJvm {
    public static void main(String[] args) {
        int a = 10;
        int b = 20;
        int sum = a + b;

        System.out.println(sum);
    }
}
```

Run:

```
javac HelloJvm.java
java HelloJvm
javap -c HelloJvm
```

Goal: see that Java source becomes `.class` bytecode.

Exercise 2: Stack Method Calls

Create a program with several method calls:

```
public class StackDemo {
    public static void main(String[] args) {
        first();
    }

    static void first() {
        second();
    }
```

```
    static void second() {
        third();
    }

    static void third() {
        System.out.println("Inside third method");
    }
}
```

Then modify `third()` to throw an exception:

```
throw new RuntimeException("Testing stack trace");
```

Goal: read the stack trace and understand method call order.

Exercise 3: Cause A StackOverflowError

```
public class StackOverflowDemo {
    public static void main(String[] args) {
        recurse();
    }

    static void recurse() {
        recurse();
    }
}
```

Run it and observe:

```
StackOverflowError
```

Goal: understand that the stack stores method calls, and too many nested calls overflow it.

Exercise 4: Heap Object References

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
    }
}

public class HeapReferenceDemo {
    public static void main(String[] args) {
        Student s1 = new Student("Asha");
        Student s2 = s1;

        s2.name = "Ravi";
```

```
        System.out.println(s1.name);
    }
}
```

Goal: understand that `s1` and `s2` point to the same heap object.

Exercise 5: Create Many Objects

```
import java.util.ArrayList;
import java.util.List;

public class HeapGrowthDemo {
    public static void main(String[] args) {
        List<byte[]> data = new ArrayList<>();

        while (true) {
            data.add(new byte[1024 * 1024]);
            System.out.println("Added 1 MB");
        }
    }
}
```

Run with limited heap:

```
java -Xmx32m HeapGrowthDemo
```

Goal: observe `OutOfMemoryError`.

Exercise 6: Observe Garbage Collection

```
public class GarbageCollectionDemo {
    public static void main(String[] args) {
        for (int i = 0; i < 1_000_000; i++) {
            String value = new String("Temporary object " + i);
        }

        System.out.println("Done");
    }
}
```

Run with GC logging:

```
java -Xlog:gc GarbageCollectionDemo
```

Goal: see that Java creates objects and the JVM cleans up unreachable ones.

Exercise 7: Static Initialization

```
class Config {
    static {
        System.out.println("Config class initialized");
    }

    static String appName = "JVM Demo";
}

public class ClassLoadingDemo {
    public static void main(String[] args) {
        System.out.println("Main started");
        System.out.println(Config.appName);
    }
}
```

Goal: observe when the class is initialized.

Exercise 8: JIT Warm-Up Observation

```
public class JitDemo {
    public static void main(String[] args) {
        long start = System.nanoTime();

        long total = 0;
        for (int i = 0; i < 500_000_000; i++) {
            total += i;
        }

        long end = System.nanoTime();

        System.out.println(total);
        System.out.println("Time: " + (end - start) / 1_000_000 + " ms");
    }
}
```

Run it multiple times.

Goal: understand that JVM performance involves runtime optimization.

Lab: JVM Architecture And Runtime Model

Lab Goal

By the end, you should be able to demonstrate:

```
Java source code -> bytecode -> class loading -> stack/heap memory -> execution -> garbage
collection
```

Prerequisites

Make sure Java is installed:

```
java -version
javac -version
```

Create a folder:

```
mkdir module1-jvm-lab
cd module1-jvm-lab
```

Task 1: Compile Java Into Bytecode

Create `HelloJvm.java`:

```
public class HelloJvm {
    public static void main(String[] args) {
        int price = 100;
        int tax = 18;
        int total = price + tax;

        System.out.println("Total: " + total);
    }
}
```

Compile and run:

```
javac HelloJvm.java
java HelloJvm
```

Inspect bytecode:

```
javap -c HelloJvm
```

Observe that Java does not directly run your `.java` file. It compiles it into a `.class` file containing bytecode.

Task 2: Observe The Stack

Create `StackTraceDemo.java`:

```
public class StackTraceDemo {
    public static void main(String[] args) {
        startOrder();
    }

    static void startOrder() {
        validateOrder();
    }

    static void validateOrder() {
```

```
        calculateTotal();
    }

    static void calculateTotal() {
        throw new RuntimeException("Something went wrong while calculating total");
    }
}
```

Compile and run:

```
javac StackTraceDemo.java
java StackTraceDemo
```

Look at the stack trace. Notice the method call chain:

```
calculateTotal
validateOrder
startOrder
main
```

This shows how the JVM tracks method calls on the stack.

Task 3: Create A StackOverflowError

Create `StackOverflowDemo.java` :

```
public class StackOverflowDemo {
    public static void main(String[] args) {
        callAgain();
    }

    static void callAgain() {
        callAgain();
    }
}
```

Run:

```
javac StackOverflowDemo.java
java StackOverflowDemo
```

Expected result:

```
StackOverflowError
```

This happens because every method call needs stack space, and infinite recursion keeps adding stack frames.

Task 4: Understand Heap References

Create `HeapReferenceDemo.java` :

```
class Account {
    String owner;
    double balance;

    Account(String owner, double balance) {
        this.owner = owner;
        this.balance = balance;
    }
}

public class HeapReferenceDemo {
    public static void main(String[] args) {
        Account account1 = new Account("Maya", 500.00);
        Account account2 = account1;

        account2.balance = 900.00;

        System.out.println(account1.owner);
        System.out.println(account1.balance);
    }
}
```

Run:

```
javac HeapReferenceDemo.java
java HeapReferenceDemo
```

Expected output:

```
Maya
900.0
```

Explanation: `account1` and `account2` point to the same object in heap memory.

Task 5: Cause An OutOfMemoryError

Create `HeapMemoryDemo.java`:

```
import java.util.ArrayList;
import java.util.List;

public class HeapMemoryDemo {
    public static void main(String[] args) {
        List<byte[]> memory = new ArrayList<>();

        while (true) {
            memory.add(new byte[1024 * 1024]);
            System.out.println("Added 1 MB");
        }
```

```
    }
}
```

Compile:

```
javac HeapMemoryDemo.java
```

Run with limited heap:

```
java -Xmx32m HeapMemoryDemo
```

Expected result:

```
OutOfMemoryError: Java heap space
```

This happens because the program keeps storing references to large objects, so the garbage collector cannot remove them.

Task 6: Observe Garbage Collection

Create `GarbageCollectionDemo.java`:

```
public class GarbageCollectionDemo {
    public static void main(String[] args) {
        for (int i = 0; i < 5_000_000; i++) {
            String value = "Object number " + i;
        }

        System.out.println("Finished creating temporary objects");
    }
}
```

Compile:

```
javac GarbageCollectionDemo.java
```

Run with GC logging:

```
java -Xlog:gc GarbageCollectionDemo
```

If your Java version does not support that flag, try:

```
java -verbose:gc GarbageCollectionDemo
```

Goal: notice that temporary objects can be cleaned up because they are no longer reachable.

Task 7: Observe Class Initialization

Create `ClassLoadingDemo.java`:

```
class AppSettings {
    static {
        System.out.println("AppSettings class initialized");
    }

    static String applicationName = "Banking App";
}

public class ClassLoadingDemo {
    public static void main(String[] args) {
        System.out.println("Main method started");
        System.out.println(AppSettings.applicationName);
    }
}
```

Run:

```
javac ClassLoadingDemo.java
java ClassLoadingDemo
```

Expected output:

```
Main method started
AppSettings class initialized
Banking App
```

This shows that classes are initialized when they are first actively used.

Lab Challenge

Create a class named `JvmStory.java` that demonstrates all of these in one program:

```
1. A main method
2. At least three method calls
3. One object created with new
4. Two references pointing to the same object
5. A static block
6. A printed explanation of what is on the stack and heap
```

Reflection Questions

Answer these after completing the lab:

1. What is the difference between `.java` and `.class` files?
2. What happens when a method is called?
3. Where are objects stored?
4. Why did `account1.balance` change when you modified `account2.balance`?

5. Why did the heap memory example fail?
6. What does garbage collection remove?
7. When does a static block run?

Module 1 Key Takeaway

```
Java source becomes bytecode.
Bytecode runs inside the JVM.
The JVM loads classes, manages memory, executes code, optimizes hot paths, and cleans unused objects.
```